Opcodes: 39	opcode (5-bit) group	funct4 (4-bit) operation	funct3 (3-bit) variant	rd (4-bit)	rs1 (4-bit)	rs2 (4-bit)	imm[bits]	Description	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
NOP	0x0	0x0	0x0	-	-	-	-	No Operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20
HALT	0x0	0x1	0x0	-	-	-	-	Halt (stop or wait for interrupt)	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20
ADD	0x1	0x0	0x0	rd	rs1	rs2	-	rd = rs1 + rs2 — Adds contents of registers rs1 and rs2, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
SUB	0x1	0x1	0x0	rd	rs1	rs2	-	rd = rs1 - rs2 — Subtracts rs2 from rs1, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
MUL	0x1	0x2	0x0	rd	rs1	rs2	-	rd = rs1 * rs2 — Multiplies rs1 by rs2, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
DIV	0x1	0x3	0x0	rd	rs1	rs2	-	rd = rs1 / rs2 — Divides rs1 by rs2, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
ADDI	0x1	0x0	0x1	rd	rs1	-	imm[12]	rd = rs1 + imm — Adds immediate 12-bit signed imm to rs1, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SUBI	0x1	0x1	0x1	rd	rs1	-	imm[12]	rd = rs1 - imm — Subtracts immediate 12-bit signed imm from rs1, stores in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
MULI	0x1	0x2	0x1	rd	rs1	-	imm[12]	rd = rs1 * imm — Multiplies rs1 by immediate imm, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
DIVI	0x1	0x3	0x1	rd	rs1	-	imm[12]	rd = rs1 / imm — Divides rs1 by immediate imm, stores result in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
LUI	0x2	0x0	0x0	rd	-	-	imm[16]	Higher Load Upper Immediate — loads upper bits of a 16-bit immediate, usually for high part of an address.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16
AUIPC	0x2	0x1	0x0	rd	-	-	imm[16]	PC + (imm[16] << 0) — Load Address Upper Immediate PC, used for position-independent code.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16
SB	0x3	0x0	0x0	-	rs1	rs2	imm[12]	Stores byte (8 bits) from rs2 to MEM[rs1 + imm].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SH	0x3	0x1	0x0	-	rs1	rs2	imm[12]	Stores halfword (16 bits) from rs2 to MEM[rs1 + imm].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SW	0x3	0x2	0x0	-	rs1	rs2	imm[12]	Stores word (32 bits) from rs2 to MEM[rs1 + imm].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BEQ	0x4	0x0	0x0	-	rs1	rs2	imm[12]	Branch if rs1 == rs2, PC += imm[12] (relative jump).	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BNE	0x4	0x1	0x0	-	rs1	rs2	imm[12]	Branch if rs1 != rs2, PC += imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BLT	0x4	0x2	0x0	-	rs1	rs2	imm[12]	Branch if rs1 < rs2 (signed), PC += imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BGT	0x4	0x3	0x0	-	rs1	rs2	imm[12]	Branch if rs1 > rs2 (signed), PC += imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BLE	0x4	0x4	0x0	-	rs1	rs2	imm[12]	Branch if rs1 <= rs2 (signed), PC += imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
BGE	0x4	0x5	0x0	-	rs1	rs2	imm[12]	Branch if rs1 >= rs2 (signed), PC += imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
JAL	0x5	0x1	0x0	rd	-	-	imm[16]	Jump and link: saves PC+4 in rd, jumps to rs1 + imm[12].	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16	i16
JALR	0x5	0x2	0x0	rd	rs1	-	imm[12]	Jump and link register: jumps to rs1 + imm[12], saves return address in rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
LW	0x6	0x0	0x0	rd	rs1	-	imm[12]	rd = MEM[rs1 + imm] — Loads 32-bit word from memory at address rs1 + imm[16] into register rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
LH	0x6	0x1	0x0	rd	rs1	-	imm[12]	rd = MEM[rs1 + imm] — Loads 16-bit half word from memory at address rs1 + imm[16] into register rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
LB	0x6	0x2	0x0	rd	rs1	-	imm[12]	rd = MEM[rs1 + imm] — Loads 8-bit byte from memory at address rs1 + imm[16] into register rd.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
AND	0x7	0x0	0x0	rd	rs1	rs2	-	AND bitwise operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
OR	0x7	0x1	0x0	rd	rs1	rs2	-	OR bitwise operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
XOR	0x7	0x2	0x0	rd	rs1	rs2	-	XOR bitwise operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
ANDI	0x7	0x0	0x1	rd	rs1	-	imm[12]	AND with Immediate Bitwise Operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
ORI	0x7	0x1	0x1	rd	rs1	-	imm[12]	OR with Immediate Bitwise Operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
XORI	0x7	0x2	0x1	rd	rs1	-	imm[12]	XOR with Immediate Bitwise Operation	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SLL	0x8	0x0	0x0	rd	rs1	rs2	-	Shift Left Logical - Shifts the bits in rs1 left by the amount in rs2. Fills in zeros from the right.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
SRL	0x8	0x1	0x0	rd	rs1	rs2	-	Shift RIght Logical - Shifts bits in rs1 right by amount in rs2. Fills in zeros from the left.	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	r3	r3	r3	r3	p8	p8	p8	p8	p8	p8	p8	p8
SLLI	0x8	0x0	0x1	rd	rs1	-	imm[12]	Shift Left Logical Immediate	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SRLI	0x8	0x0	0x1	rd	rs1	-	imm[12]	Shift RIght Logical Immediate	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	r2	r2	r2	r2	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12	i12
SYS	0x9	0x0	0x0	-	-	-	imm[8]	System Calls	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	i8	i8	i8	i8	i8	i8	i8	i8	p12	p12	p12	p12	p12	p12	p12	p12	p12	p12	p12	p12
PUSH	0xA	0x0	0x0	-	rs1	-	-	Push unto stack	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16
POP	0xA	0x1	0x0	rd	-	-	-	Pops from stack	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	r1	r1	r1	r1	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16	p16
CALL	0xA	0x2	0x0	-	-	-	imm[20]	Call subroutine at target	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20	i20
RET	0xA	0x3	0x0	-	-	-	-	Returns from subroutine	op	op	op	op	op	f4	f4	f4	f4	f3	f3	f3	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20	p20